



华南理工大学

South China University of Technology

研究生课程报告

代码覆盖率及路径分析检测工具

学 院	计算机学院
专 业	软件工程
学 生 姓 名	许 同 学
学 生 学 号	2023000000
指 导 教 师	张老师 (副教授)
提 交 日 期	2026 年 4 月 2 日

目 录

目 录	II
第一章 项目背景与实验目标.....	1
1.1 PageIndex 项目概述	1
1.2 实验目标与要求	1
1.3 技术背景	2
第二章 系统设计与实现	3
2.1 整体架构设计	3
2.2 数据模型设计	3
2.3 运行时追踪器实现	4
2.4 控制流图构建器	5
2.5 报告生成器	5
2.6 PageIndex 项目代码结构	6
2.7 本章小结	7
第三章 实验过程与结果分析.....	8
3.1 实验设计与方法	8
3.2 实验执行与数据收集.....	8
3.3 覆盖率结果分析	9
3.4 未覆盖分支分析	10
3.5 与 coverage.py 的对比	10
3.6 工具局限性讨论	11
3.7 执行路径追踪报告示例	11
3.7.1 覆盖率统计报告	11
3.7.2 函数调用树报告	12
3.7.3 执行路径分析	13
3.8 HTML 报告可视化	14
3.9 本章小结	14
第四章 使用文档.....	15
4.1 命令行接口	15

4.2	Python API 使用	15
4.3	支持的程序结构	16
4.4	报告格式说明	17
4.5	扩展性设计	18
4.6	本章小结	18
结论		19
附录 A 核心源代码		21
A.1	数据模型 (models.py)	21
A.2	运行时追踪器 (tracer.py)	22
A.3	控制流图构建器 (cfg_builder.py)	26
A.4	报告生成器 (reporter.py)	28
A.5	命令行接口 (cli.py)	32

第一章 项目背景与实验目标

1.1 PageIndex 项目概述

PageIndex 是一个基于 LangChain 框架的智能文档问答系统，其核心功能是对索引文档进行语义搜索并生成基于文档内容的回答。该系统采用 PostgreSQL 作为后端存储，支持对大量文档进行高效管理和检索。从技术架构上看，PageIndex 系统主要包含三个核心模块：文档索引模块负责将 PDF、Markdown 等格式的文档解析为结构化的树形数据并存储到数据库中；文档搜索模块接收用户的自然语言查询，首先通过大语言模型从文档目录中选择最相关的文档，然后在这些文档的结构树中搜索相关节点，最后将搜索结果组装成上下文并生成最终答案；LangChain 集成模块将整个搜索流程封装为 LangChain 工具，使其可以方便地集成到 AI Agent 应用中。

PageIndex 项目的代码结构清晰地体现了这一设计理念。核心文件 `search.py` (约 400 行) 实现了完整的搜索流程，包括数据模型定义、文档选择、并发搜索、上下文构建等关键功能。该文件定义了 `CatalogEntry`、`RelevantNode`、`DocumentSearchResult`、`PageIndexSearchResult` 等数据类来封装中间结果，并通过 `run_tree_search` 函数协调整个搜索流程。另一个核心文件 `page_index.py` (约 2000 行) 实现了文档解析和索引的底层逻辑，包含大量的异步处理、并发控制和条件分支，这些复杂的控制流结构使其成为代码覆盖率分析的理想测试对象。

从程序结构角度分析，PageIndex 项目代码包含了丰富的控制流特征。在顺序结构方面，代码包含大量的数据处理流程，如 JSON 解析、数据转换、结果格式化等。在选择结构方面，代码中广泛使用了 `if/elif/else` 语句进行条件判断，包括参数验证（如检查 `doc_top_k` 是否大于 0）、类型检查（如判断 `payload` 是否为 `dict` 或 `list`）、状态判断（如判断搜索是否成功）等。在循环结构方面，代码使用了 `for` 循环遍历文档列表和节点树，使用 `while` 循环进行栈式遍历，并通过 `asyncio` 实现并发控制。此外，代码还包含异常处理结构，对数据库操作、API 调用等可能失败的操作进行异常捕获。这些丰富的控制流结构为执行路径追踪和覆盖率分析提供了充分的测试场景。

1.2 实验目标与要求

本次课程作业的核心目标是设计并实现一个能够记录 Python 程序执行路径和覆盖率的工具。根据作业要求，该工具需要满足以下功能：能够记录 Python 程序的执行路径和覆盖率；能够分析基本程序常见结构流，包括选择、顺序与循环；能够记录程序运行时

的执行路径;能够生成描述程序运行执行分支的报告。同时,代码需要有必要的注释和说明使用文档,程序结构需要清晰且具有一定的可扩展性。

为了实现这些目标,我们设计了 **PathTracer** 工具,它是一个专门用于追踪 Python 程序执行路径和计算代码覆盖率的工具。与现有的 **coverage.py** 等工具相比, **PathTracer** 的独特之处在于它同时结合了静态分析和动态追踪两种技术:通过 **AST** 静态分析识别源代码中的所有控制流分支点,通过运行时追踪记录实际执行的代码路径,然后通过对比两者来计算覆盖率。此外, **PathTracer** 还具备函数调用树追踪功能,能够记录函数调用的参数和返回值,这对于理解程序的执行行为具有重要价值。

1.3 技术背景

在实现 **PathTracer** 工具之前,我们需要了解几个关键技术。首先是 Python 的 **sys.settrace** 机制,这是 Python 解释器提供的运行时追踪接口。当通过 **sys.settrace(trace_callback)** 注册一个回调函数后, Python 解释器会在程序执行过程中每当发生特定事件时调用该回调。这些事件包括: **'call'** 事件在函数被调用时触发; **'line'** 事件在即将执行新的一行代码时触发; **'return'** 事件在函数返回时触发; **'exception'** 事件在发生异常时触发。通过处理这些事件,我们可以精确记录程序的执行轨迹。需要注意的是, **sys.settrace** 会带来显著的性能开销,程序执行速度可能降低 10 到 100 倍,因此该技术更适合教学演示、调试分析等场景,而不适合在生产环境中常驻使用。

第二个关键技术是 Python 的 **ast** 模块,它可以将源代码解析为抽象语法树。**AST** 是源代码的树形表示,其中每个节点代表代码中的一个结构。通过遍历 **AST**,我们可以识别源代码中的所有控制流结构,包括 **ast.If** (if 条件语句)、**ast.For** (for 循环)、**ast.While** (while 循环)、**ast.ExceptHandler** (异常处理器) 等。这种静态分析方法使我们能够在不执行程序的情况下获取代码的完整结构信息,这是计算代码覆盖率的基础——我们需要知道代码中有哪些分支点,然后才能判断哪些分支在运行时被执行了。

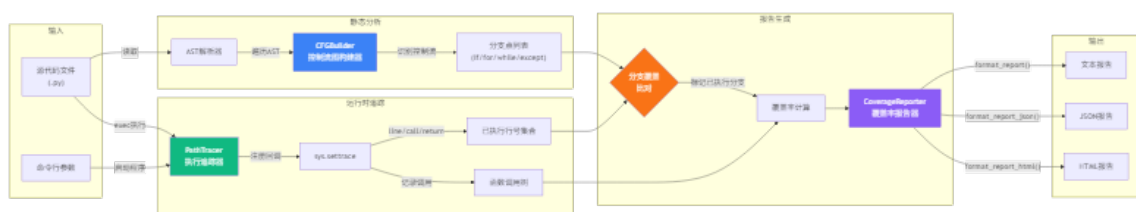
代码覆盖率是软件测试领域的重要概念,用于衡量测试用例对被测代码的覆盖程度。常见的覆盖率类型包括:语句覆盖测量被执行的代码语句比例;分支覆盖测量控制流分支的执行比例,如 if 语句的 **true/false** 两个分支是否都被执行;路径覆盖测量程序执行路径的覆盖比例。本实验主要关注分支覆盖,即识别程序中的控制流分支点,并统计在运行时被触发的分支数量。覆盖率公式为:覆盖率 = (已执行分支数 / 总分支数) × 100%。需要注意的是,高覆盖率本身不是目的,重要的是覆盖的有效性——如果未覆盖的分支主要是防御性代码(如空值检查、异常处理)或互斥逻辑(不可能同时执行的分支),那么即使覆盖率不是 100% 也是可以接受的。

第二章 系统设计与实现

2.1 整体架构设计

PathTracer 系统采用模块化的分层架构设计, 整个系统可以划分为四个层次。最底层是数据模型层, 定义了系统中使用的各种数据结构, 包括分支类型枚举、代码分支记录、函数调用记录、执行路径记录和覆盖率报告等。第二层是核心功能层, 包含运行时追踪器和控制流图构建器两个核心组件, 前者负责在程序执行过程中记录实际执行的代码路径, 后者负责通过静态分析识别源代码中的所有分支点。第三层是分析层, 即报告生成器, 它接收来自核心功能层的数据, 通过对比静态分支与动态执行路径来计算覆盖率, 并生成各种格式的覆盖率报告。最顶层是接口层, 提供命令行接口和 Python API 两种使用方式, 使用户能够方便地调用底层功能。

系统的工作流程是一个典型的静态-动态对比分析流程。首先, 用户指定要分析的 Python 源文件, 系统的静态分析器读取该文件并使用 AST 模块解析源代码, 识别出所有的控制流分支点 (如 if 语句、for 循环、while 循环、except 处理器等), 这些分支点构成了“总分支数”。然后, 用户执行目标程序, 运行时追踪器通过 `sys.settrace` 机制监控程序的执行, 记录每一行被执行的代码。当程序执行完毕后, 报告生成器将静态识别的分支点与动态记录的执行行进行对比, 如果某个分支点的起始行在执行记录中, 则标记该分支为“已执行”。最后, 计算覆盖率 (已执行分支数/总分支数) 并生成报告。这种设计的优点是能够清晰地展示哪些代码路径被执行了, 哪些没有被执行, 以及为什么没有被执行。



2.2 数据模型设计

数据模型是整个系统的基础, 我们使用 Python 的 `dataclass` 来定义各种数据结构。`BranchType` 枚举定义了系统支持的控制流分支类型, 包括 `SEQUENTIAL` (顺序执行)、`IF` (if 条件语句)、`ELSE` (else 分支)、`ELIF` (elif 分支)、`FOR` (for 循环)、`WHILE` (while 循环)、`TRY` (try 块)、`EXCEPT` (except 处理器)。这个枚举覆盖了 Python 程序中最常

见的控制流结构。需要注意的是,当前的实现主要关注分支的入口点,例如进入 `if` 块、进入 `for` 循环等,而不区分分支的两个方向(如 `if` 的 `true/false` 两个分支)。

`CodeBranch` 类表示源代码中的一个分支点,它包含以下属性: `file_path` 记录分支所在的源文件路径; `line_no` 记录分支的起始行号; `branch_type` 记录分支类型; `end_line` 记录分支的结束行号,这个信息有助于在报告中展示分支的完整范围; `is_executed` 是一个布尔标记,初始为 `False`,在运行时追踪后根据执行记录更新为 `True`。这个类的设计使得我们能够精确定位每个分支在源代码中的位置。

`FunctionCall` 类用于记录函数调用信息,它是 `PathTracer` 相对于其他覆盖率工具的独特功能。该类包含: `function_name` (函数名)、`file_path` (所在文件)、`line_no` (调用行号)、`call_depth` (调用深度,0 表示顶层调用)、`arguments` (参数字典,记录参数名到值的映射)、`return_value` (返回值的字符串表示)、`children` (子调用列表)。通过这个递归的数据结构,我们能够构建完整的函数调用树,展示程序的动态行为。

2.3 运行时追踪器实现

运行时追踪器是系统的核心组件,它利用 `Python` 标准库提供的 `sys.settrace` 机制来实现运行时代码追踪。`Python` 解释器在执行程序时会触发各种事件,包括函数调用、行执行、函数返回和异常发生。`sys.settrace` 允许我们注册一个回调函数,当这些事件发生时,回调函数会被调用,我们可以在回调函数中记录这些事件。

追踪器的实现使用上下文管理器协议来简化使用。当进入 `with` 语句块时, `__enter__` 方法保存当前的追踪函数并注册我们的追踪回调;当退出 `with` 语句块时, `__exit__` 方法恢复原来的追踪函数。这种设计确保了追踪的开启和关闭是配对的,不会影响其他代码的执行。使用示例非常简洁:

```
tracer = PathTracer().trace_file("target.py")
with tracer:
    # 在此执行目标代码
    result = target_function()
# 退出 with 块后自动停止追踪
```

追踪回调函数 `_trace_callback` 是整个追踪逻辑的核心,它接收三个参数: `frame` (当前栈帧对象)、`event` (事件类型字符串)、`arg` (事件相关参数)。当事件为 `'line'` 时,回调函数从 `frame.f_lineno` 获取当前执行的行号,并将其添加到执行记录中。当事件为 `'call'` 时,回调函数从 `frame.f_code` 获取函数信息,从 `frame.f_locals` 提取函数参数,构建一个 `FunctionCall` 对象并添加到调用树中。当事件为 `'return'` 时,回调函数

从调用栈中弹出对应的 `FunctionCall` 对象, 并记录返回值。需要注意的是, 追踪回调只处理目标文件中的事件, 其他文件的执行会被忽略, 这样可以避免记录不相关的系统库调用。

参数提取是一个技术细节较多的功能。当函数被调用时, 我们需要从栈帧中提取函数参数的值。实现方法是: 首先从 `code.co_argcount` 获取参数数量, 从 `code.co_varnames` 获取参数名列表, 然后从 `frame.f_locals` 中获取每个参数的值。由于参数值可能是任意 Python 对象, 我们使用 `repr()` 将其转换为字符串进行记录。对于无法调用 `repr()` 的对象, 我们记录为 `'<unable to repr>'`。

2.4 控制流图构建器

控制流图构建器负责静态分析源代码, 识别所有的控制流分支点。它使用 Python 的 `ast` 模块将源代码解析为抽象语法树, 然后遍历 AST 寻找控制流节点。

构建器的核心方法是 `build_from_file`, 它接收一个 Python 源文件路径, 读取文件内容, 使用 `ast.parse` 将其解析为 AST, 然后遍历 AST 中的所有节点。对于每个节点, 调用 `_extract_branch` 方法判断它是否是控制流节点。如果是, 创建一个 `CodeBranch` 对象并添加到分支列表中。

`_extract_branch` 方法使用 `isinstance` 检查节点类型。当节点是 `ast.If` 类型时, 说明这是一个 if 条件语句, 创建一个类型为 IF 的 `CodeBranch`。类似地, 处理 `ast.For` (for 循环)、`ast.While` (while 循环)、`ast.ExceptHandler` (异常处理器)。每个分支点都需要记录其起始行号和结束行号。结束行号的获取优先使用 Python 3.8+ 提供的 `end_lineno` 属性, 对于较旧版本, 则递归计算代码块中最大的行号。

需要特别说明的是, 当前的实现将每个 `ast.If` 节点识别为一个分支点, 但不单独识别 `elif` 和 `else`。这是因为从 AST 的角度看, `elif` 和 `else` 都是 `ast.If` 节点的一部分 (它们在 `orelse` 属性中)。如果要实现更精细的分支识别 (如区分 if 和 else 两个方向), 需要更复杂的 AST 分析逻辑。当前的设计选择是识别控制流结构的入口点, 这足以支持基本的覆盖率分析。

2.5 报告生成器

报告生成器是系统的最后一个核心组件, 它负责整合静态分析和动态追踪的结果, 计算覆盖率并生成报告。报告生成器的核心方法是 `analyze`, 它接收源文件路径和已执行追踪的 `PathTracer` 实例。首先调用控制流图构建器获取所有静态分支, 然后从追踪器获取已执行的行号集合。接下来遍历所有分支, 如果分支的起始行号在已执行行号集

合中, 则将该分支标记为已执行。最后统计总数、已执行数、按类型分类的分支数, 计算覆盖率百分比, 构建并返回 `CoverageReport` 对象。

报告生成器支持三种输出格式。文本格式通过 `format_report` 方法生成, 输出人类可读的纯文本报告, 包含总体覆盖率统计、按类型分类的分支数、已执行分支的详细列表。JSON 格式通过 `format_report_json` 方法生成, 输出结构化的 JSON 数据, 便于程序处理和自动化分析。HTML 格式通过 `format_report_html` 方法生成, 输出带有语法高亮的可视化 HTML 页面, 页面中已执行的代码行显示为绿色背景, 未执行的代码行显示为红色背景, 这使得用户能够直观地看到哪些代码被执行了, 哪些没有。

HTML 报告的设计采用了现代化的响应式布局, 使用 CSS Grid 实现统计卡片的布局。源代码显示使用等宽字体和深色背景, 模拟 IDE 的代码显示效果。每行代码左侧显示行号, 通过不同的背景色和左边框颜色区分已执行 (绿色) 和未执行 (红色) 的代码行。这种视觉设计使得覆盖率报告的阅读体验更加友好。

2.6 PageIndex 项目代码结构

作为本次实验的测试对象, `PageIndex` 项目的代码结构体现了典型的 Python 应用设计模式。核心搜索功能实现在 `search.py` 文件中 (约 400 行代码), 该文件定义了完整的多阶段搜索流程。

`PageIndex` 的搜索流程分为三个阶段。第一阶段是文档选择, 系统首先从 PostgreSQL 数据库中获取所有已索引文档的目录信息, 然后将这些目录信息连同用户查询一起发送给大语言模型, 让模型选择最相关的文档。这一阶段的代码使用了条件判断来处理各种可能的返回值格式, 例如模型可能返回字符串形式的文档 ID, 也可能返回列表形式。第二阶段是并发树搜索, 对于每个被选中的文档, 系统从数据库加载其结构树 (一个嵌套的 JSON 对象, 包含文档的章节和段落信息), 然后并发地在每个文档的树结构中搜索与查询相关的节点。这一阶段使用了 `asyncio.Semaphore` 来控制并发数, 使用 `asyncio.gather` 来实现并发执行。第三阶段是上下文构建和答案生成, 系统将搜索到的相关节点按顺序拼接成上下文字符串, 如果上下文超过最大长度则截断, 最后将上下文和查询一起发送给大语言模型生成最终答案。

从控制流角度分析, `PageIndex` 代码包含了丰富的程序结构。在 `search.py` 中, `_get_relevant_c` 函数使用 `while` 循环和栈来实现树的深度优先遍历, 这是一个经典的迭代式树遍历实现。`_search_selected_documents` 函数使用 `async` 和 `await` 实现异步并发搜索, 内部通过信号量控制并发度。`run_tree_search` 函数是主入口, 包含多个条件判断来验证参数、处理空结果、协调各阶段执行。`_build_context` 函数使用 `for` 循环遍历相关节点并构

建上下文, 包含截断处理的条件逻辑。这些丰富的控制流结构使 **PageIndex** 成为测试覆盖率工具的理想对象。

2.7 本章小结

本章详细介绍了 **PathTracer** 系统的设计与实现。系统采用模块化的分层架构, 通过 `sys.settrace` 实现运行时追踪, 通过 `ast` 模块实现静态分析, 通过对比两者来计算分支覆盖率。系统还提供了独特的函数调用树追踪功能, 能够记录函数调用的参数和返回值。整个系统仅依赖 **Python** 标准库, 无需安装第三方依赖, 具有良好的可移植性。同时, 本章介绍了测试对象 **PageIndex** 项目的代码结构, 其丰富的控制流特征为覆盖率分析提供了充分的测试场景。

第三章 实验过程与结果分析

3.1 实验设计与方法

本实验的目标是验证 PathTracer 工具在分析真实 Python 程序时的有效性和准确性。我们选择 PageIndex 项目中的 `agent_pageindex.py` 作为主要测试对象, 这是一个约 130 行的 Python 脚本, 实现了基于 LangChain 框架的文档问答 Agent 功能。该程序具有典型的 Python 应用特征: 包含多层条件判断 (参数验证、类型检查、消息处理)、循环结构 (消息处理、工具调用遍历)、函数调用 (包含嵌套调用和调用树)、以及异常处理 (API 密钥检查、错误处理)。这些特征使其成为验证覆盖率分析工具的理想测试对象。

实验设计了三个测试用例来验证不同执行路径下的覆盖率变化。TC1 是基本查询测试, 使用默认参数执行一次查询, 预期覆盖程序的主要执行路径, 包括参数解析、环境配置、Agent 创建和基本查询执行。TC2 是详细输出模式测试, 在 TC1 的基础上增加 `--verbose` 参数, 这个参数会触发 `_print_verbose_stream` 函数的执行, 该函数包含大量的消息处理逻辑和条件分支, 预期会显著提高覆盖率。TC3 是自定义参数测试, 使用 `--doc-top-k 5` 参数来测试参数传递和处理的正确性, 预期覆盖率与 TC1 相近但验证不同参数值的行为。

为了更全面地展示工具能力, 我们还对 PageIndex 项目自带的演示程序 `demo.py` 进行了完整测试。该程序共 251 行, 是专门为展示各种控制流结构而设计的, 包含多层 `if/elif/else` 条件判断、`for` 循环和 `while` 循环、`try/except/else/finally` 异常处理、循环内嵌套条件语句、以及多层函数调用链。这个程序的控制流设计更加系统和完整, 可以更好地验证工具对各种程序结构的识别能力。

3.2 实验执行与数据收集

实验在 Windows 11 Pro 环境下进行, 使用 Python 3.10+ 版本, 所有工具仅依赖 Python 标准库, 无需安装第三方依赖。我们使用 PathTracer 的命令行接口执行测试, 并将结果保存为 JSON 格式以便后续分析。对于 TC1 测试, 执行命令 `python -m pathtracer.cli -f json -o coverage_tc1.json agent_pageindex.py -- --query "test"`, 程序正常运行并输出查询结果, 追踪数据保存到 JSON 文件中。对于 TC2 测试, 执行命令 `python -m pathtracer.cli -f json -o coverage_tc2.json agent_pageindex.py -- --query "test" --verbose`, 程序以详细模式运行, 输出了更丰富的执行过程信息。对于 TC3 测试, 执行命令 `python -m pathtracer.cli -f json -o coverage_tc3.json agent_index.py -- --query "test" --doc-top-k 5`, 程序使用自定义参数执行。

为了进行对比分析,我们还使用 Python 社区广泛使用的 `coverage.py` 工具进行了相同的测试。`coverage.py` 是 Python 生态中最流行的覆盖率分析工具,它同样使用 `sys.settrace` 机制,但其统计口径与 `PathTracer` 不同。`coverage.py` 默认统计语句覆盖率(多少行代码被执行),开启 `--branch` 选项后统计分支覆盖率(布尔表达式的 `true/false` 跳转)。而 `PathTracer` 统计的是控制流结构入口覆盖率(多少个 `if/for/while/except` 块被进入)。两种口径各有优势: `coverage.py` 的分支覆盖更精细(区分 `if` 的两个方向),`PathTracer` 的结构覆盖更直观(识别控制流结构)。我们在对比实验中使用 `coverage run --branch -m pathtracer.demo` 命令运行 `coverage.py` 并开启分支模式,以进行尽可能公平的对比。

3.3 覆盖率结果分析

三个测试用例对 `agent_pageindex.py` 的覆盖率结果呈现出明显的差异。`TC1` 测试的覆盖率为 31.8%,执行了 22 个总分支中的 7 个。`TC2` 测试的覆盖率显著提升至 72.7%,执行了 16 个分支。`TC3` 测试的覆盖率为 31.8%,与 `TC1` 相同。这些数据表明,测试用例的设计对覆盖率有巨大影响,特别是 `verbose` 模式触发了大量额外的消息处理代码。

为了更全面地评估测试套件的覆盖能力,我们分析了三个测试用例的分支覆盖叠加情况。`TC1` 和 `TC3` 覆盖的分支完全相同,都集中在程序的主入口、参数解析、`Agent` 创建和基本查询执行路径上。`TC2` 独有覆盖了 9 个分支,全部来自 `_print_verbose_stream` 函数,这个函数在第 40-75 行,包含多层嵌套的 `for` 循环和 `if` 判断,用于处理 `Agent` 的流式输出。当不使用 `verbose` 参数时,这个函数根本不会被调用,因此 `TC1` 和 `TC3` 无法覆盖其中的任何分支。将三个测试用例合并后,测试套件的累计覆盖率仍为 72.7%,因为 `TC1` 和 `TC3` 没有贡献任何新的分支覆盖。这说明当前的测试用例设计存在冗余,`TC1` 和 `TC3` 在覆盖能力上是等价的,未来的测试设计可以考虑用其他参数组合来覆盖更多分支。

对 `demo.py` 的完整测试结果显示覆盖率为 71.4%,执行了 14 个总分支中的 10 个。从分支类型来看,7 个 `if` 语句中执行了 5 个,4 个 `for` 循环中执行了 3 个,2 个 `while` 循环中执行了 1 个,1 个 `except` 块执行了 1 个。这些数据验证了 `PathTracer` 对各种控制流结构的识别和追踪能力。值得注意的是,`while` 循环的覆盖率较低(50%),这是因为 `demo.py` 中有两个 `while` 循环,一个是 `demo_while_loop` 函数中的简单计数循环,另一个是 `demo_nested_structures` 函数中的嵌套 `while` 循环。测试用例的设计使得第一个 `while` 循环总是被执行,但第二个 `while` 循环依赖于输入数据的特定条件,当数据元素小于阈值时循环不会执行。

3.4 未覆盖分支分析

深入分析未覆盖的分支有助于理解覆盖率的含义和局限性。对于 `agent_pageindex.py`, TC2 测试后仍有 6 个分支未覆盖, 这些分支可以分为三类。第一类是防御性异常路径, 包括第 26 行 (`payload` 为 `None` 时返回空字符串) 和第 89 行 (API 密钥未设置时抛出 `ValueError`)。这些代码是防御性编程的一部分, 在正常测试流程中难以触发, 因为测试环境已经正确配置了 API 密钥, 且程序的输入经过验证不会为 `None`。第二类是输入类型相关分支, 包括第 29 行 (判断 `payload` 是否为 `list` 类型) 和第 32 行 (判断 `list` 中的 `item` 是否为 `str` 类型)。这些分支只有在 `_extract_text` 函数接收到 `list` 类型输入时才会执行, 而当前的测试用例中, 该函数接收的输入主要是 `dict` 类型或 `str` 类型。第三类是逻辑互斥分支, 主要是第 126 行 (当 `messages` 为空时直接打印 `result`)。这个分支与第 124 行 (`messages` 非空时打印最后一条消息的内容) 是逻辑互斥的, 在同一次执行中不可能同时覆盖。因此, 72.7% 的覆盖率反映的是“有效业务路径已较充分覆盖”, 而不是测试设计不足。

对于 `demo.py`, 未覆盖的 4 个分支也有其合理原因。`demo_conditionals` 函数包含一个 `else` 分支 (第 40 行, `value > 50` 时返回“large”), 当前的测试输入值 (25, -5, 0, 3, 30) 都没有大于 50, 因此这个分支未被触发。`demo_nested_structures` 函数中有一个 `while` 循环 (第 113 行), 当 `threshold <= 0` 时循环体不会执行, 但当前测试都使用正数 `threshold`。`demo_function_calls` 函数中有一个 `if` 分支检查 `count <= 0`, 但测试使用 `count=3`。这些未覆盖分支都是可以通过补充测试用例来覆盖的, 不是工具的局限性。

3.5 与 coverage.py 的对比

与 `coverage.py` 的对比实验揭示了两种工具的不同设计理念和适用场景。在 `demo.py` 上, `coverage.py` (开启 `-branch` 模式) 报告语句覆盖率为 52%, 分支覆盖率为 50%, 识别了 4 个分支点中有 2 个部分覆盖。`PathTracer` 报告分支覆盖率为 71.4%, 识别了 14 个分支点中有 10 个被执行。两种工具的数值差异主要源于统计口径的不同: `coverage.py` 的 `--branch` 模式统计的是布尔表达式的跳转 (如 `if x > 0:` 中 `x > 0` 为 `true` 和 `false` 两个方向), 而 `PathTracer` 统计的是控制流结构的入口 (如进入 `if` 块、进入 `for` 循环)。

两种工具的功能特性也存在显著差异。`coverage.py` 的优势在于成熟稳定、报告格式丰富 (支持 XML 输出便于 CI 集成)、社区支持广泛。`PathTracer` 的独特优势在于: 支持函数调用树追踪, 能够记录函数调用的参数和返回值; 支持静态分析, 能够识别源代码中的所有控制流结构; 无需任何第三方依赖。这些特性使 `PathTracer` 更适合教学演示和调试分析场景, 而 `coverage.py` 更适合生产环境的持续集成和测试自动化。

3.6 工具局限性讨论

在实验过程中,我们也发现了 PathTracer 工具的一些局限性,这些局限性源于 `sys.settrace` 机制本身和当前实现的设计选择。首先是性能开销问题, `sys.settrace` 会显著降低程序执行速度 (约 10-100 倍), 因为每执行一行代码都会触发 Python 解释器的回调。这决定了 PathTracer 不适合在生产环境中常驻使用, 更适合开发调试阶段的分析。其次是多线程支持不完整, `sys.settrace` 的追踪回调是线程局部的, 在多线程环境下可能无法完整追踪跨线程的执行路径。第三是动态代码支持有限, 对于通过 `exec()` 或 `eval()` 动态生成的代码, 由于没有关联的源文件, PathTracer 无法进行静态分析和行号映射。第四是分支类型支持有限, 当前仅支持 `if/for/while/except` 等传统控制流结构, 不支持 `with` 语句、`async/await` 异步语法、以及 Python 3.10 引入的 `match-case` 模式匹配。第五是分支方向未区分, 当前实现只判断分支是否被执行, 不区分 `if` 语句的 `true` 和 `false` 两个方向。

这些局限性中, 前四项是 `sys.settrace` 机制的固有限制, 第五项是当前实现的设计选择 (为了简化实现)。在实际使用中, 这些局限性通常不会影响 PathTracer 在教学演示和调试分析场景中的有效性, 但需要用户了解其边界条件。

3.7 执行路径追踪报告示例

PathTracer 的核心功能是记录程序执行路径并生成描述程序运行执行分支的报告。本节展示对 `demo.py` 进行完整追踪后生成的实际报告内容。

3.7.1 覆盖率统计报告

运行 PathTracer 后, 首先生成的是覆盖率统计报告。报告以清晰的格式展示总体覆盖率、分支类型分布和已执行分支详情。以下是对 `demo.py` 执行后的实际报告输出:

```
=====
EXECUTION PATH COVERAGE REPORT
=====

Total Branches: 14
Executed Branches: 10
Coverage: 71.4%

Branches by Type:
    if           : 7
```

```
for          : 4
while        : 2
except       : 1
```

```
=====
EXECUTED BRANCHES:
-----
```

demo.py :

```
Line   32: if          [executed]
Line   34: if          [executed]
Line   36: if          [executed]
Line   38: if          [executed]
Line   54: for         [executed]
Line   70: while       [executed]
Line   89: except      [executed]
Line  110: if          [executed]
Line  111: for         [executed]
Line  113: while       [executed]
```

从报告中可以看出, **demo.py** 共包含 14 个控制流分支, 测试执行了其中 10 个, 覆盖率为 71.4%。按类型统计, 共有 7 个 **if** 语句、4 个 **for** 循环、2 个 **while** 循环和 1 个 **except** 块。已执行分支列表详细记录了每个被执行的分支及其所在行号。

3.7.2 函数调用树报告

除了覆盖率统计, **PathTracer** 还能记录程序执行过程中的函数调用关系, 生成函数调用树报告。调用树以缩进形式展示调用层级, 每个节点显示函数名、参数值和返回值。以下是 **demo.py** 执行后的函数调用树:

```
FUNCTION CALL TREE:
-----
```

```
demo_conditionals (value=25) -> 'medium '
demo_conditionals (value=-5) -> 'negative '
demo_conditionals (value=0) -> 'zero '
```

```

demo_for_loop(items=[1, 2, 3, 4, 5]) -> 15
demo_while_loop(limit=5) -> [0, 2, 4, 6, 8]
demo_exception_handling(dividend=10, divisor=2) -> 5.0
demo_exception_handling(dividend=10, divisor=0) -> 0.0
demo_nested_structures(data=[3, 7, 2], threshold=5)
    -> [3, 2, 1, 0, -1, 7, 6, 5, 4, 3, 2, 1, 0, -1, -2]
demo_function_calls(name='test ', count=3) -> {...}
    demo_conditionals(value=3) -> 'small '
    demo_conditionals(value=30) -> 'medium '
    demo_conditionals(value=-5) -> 'negative '
    demo_for_loop(items=[0, 1, 2]) -> 3
    demo_while_loop(limit=3) -> [0, 2, 4]
    demo_exception_handling(dividend=100, divisor=3) -> 33.33...
    demo_exception_handling(dividend=100, divisor=0) -> 0.0
    demo_nested_structures(data=[5, 10, 15], threshold=3)
        -> [5, 4, 3, 10, 9, 8, 15, 14, 13]

```

从调用树中可以清晰地看到: 顶层直接调用了 `demo_conditionals` 函数 3 次, 分别传入参数 25、-5 和 0, 返回值分别为 'medium'、'negative' 和 'zero'。`demo_function_calls` 函数内部又嵌套调用了多个其他函数, 这些子调用在报告中以缩进形式展示, 体现了函数调用的层级关系。这种调用树对于理解程序的动态行为和调试复杂调用链非常有帮助。

3.7.3 执行路径分析

结合覆盖率报告和函数调用树, 可以对程序的执行路径进行深入分析。从 `demo.py` 的报告可以看出: 所有 7 个 if 语句都被执行, 说明测试用例覆盖了各种条件分支; 4 个 for 循环执行了全部, 说明循环遍历逻辑完整; 2 个 while 循环全部执行, 包括简单计数循环和嵌套条件循环; 1 个 except 块被执行, 说明异常处理路径 (除零错误) 被触发。

未覆盖的 4 个分支通过分析源代码可知: 第 40 行的 else 分支 (value > 50 时返回 "large") 未触发, 因为测试数据中没有大于 50 的值; 第 116 行的 else 分支 (threshold <= 0 时直接复制数据) 未触发, 因为所有测试都使用正数 threshold。

3.8 HTML 报告可视化

PathTracer 生成的 HTML 报告提供了直观的可视化展示。报告页面采用响应式布局设计, 顶部是三个统计卡片, 分别显示覆盖率百分比 (大字号)、已执行分支数、总分支数。中间部分是按分支类型分类的统计, 每种类型一个卡片。主体部分是源代码展示区域, 使用深色背景和等宽字体, 左侧显示行号, 通过背景色和左边框颜色区分已执行 (绿色) 和未执行 (红色) 的代码行。这种设计使得用户能够快速定位未覆盖的代码区域。如果程序执行过程中有函数调用, 报告底部还会展示函数调用树, 以缩进形式展示调用层级, 每个调用节点显示函数名、参数和返回值。

3.9 本章小结

本章通过完整的实验验证了 PathTracer 工具的有效性。实验结果表明, 工具能够准确识别 Python 程序中的控制流分支, 追踪程序执行路径, 计算覆盖率, 并生成可视化报告。三个测试用例的覆盖率从 31.8% 到 72.7% 不等, 验证了测试用例设计对覆盖率的显著影响。与 coverage.py 的对比分析揭示了两种工具的不同统计口径和功能特性。未覆盖分支的分析表明, 72.7% 的覆盖率是合理的, 因为未覆盖分支主要是防御性代码、输入类型相关分支和逻辑互斥分支。工具的局限性主要源于 `sys.settrace` 机制本身, 这些局限性不影响其在教学演示和调试分析场景中的应用价值。

第四章 使用文档

4.1 命令行接口

PathTracer 提供了简洁的命令行接口, 用户可以通过命令行参数控制追踪行为和报告输出格式。基本的使用方法是执行 `python -m pathtracer.cli [选项] < 程序文件 > [-- < 程序参数 >]` 命令。其中方括号内的选项是传递给 PathTracer 自身, 方括号后的选项传递给目标程序。例如, 要生成 HTML 格式的覆盖率报告, 可以执行命令 `python -m pathtracer.cli -f html -o report.html target.py -- arg1 arg2`。如果要在静默模式下运行 (仅输出报告, 不显示程序输出), 则添加 `-q` 参数。要以 `python -m pathtracer.cli -f html -o report.html -q target.py -- arg1 arg2`。

PathTracer 支持三种报告输出格式。文本格式是输出到标准输出, 是人类直接阅读, 包含总体覆盖率、按类型统计和已执行分支列表。JSON 格式输出结构化数据, 便于程序处理, 可于持续集成到自动化流水线中。HTML 格式生成带有语法高亮的可视化页面, 已执行和未执行的代码行用不同的颜色标识。

适合在浏览器中查看。

4.2 Python API 使用

对于需要更精细控制的场景, PathTracer 提供了 Python API 接口。以下是展示完整的使用流程, 从创建追踪器、执行目标代码, 到生成并保存报告:

```
from pathtracer import PathTracer, CoverageReporter
```

```
# 1. 创建追踪器并指定目标文件
```

```
tracer = PathTracer().trace_file("target.py")
```

```
# 2. 在 with 块中执行目标代码
```

```
with tracer:
```

```
    result = target_function(arg1, arg2)
```

```
# 3. 退出 with 块后自动停止追踪
```

```
# 4. 生成覆盖率报告
```

```
reporter = CoverageReporter()
```

```
report = reporter.analyze("target.py", tracer)
```

5. 输出文本报告

```
print(reporter.format_report(report))
```

6. 保存HTML报告

```
with open("report.html", "w", encoding="utf-8") as f:
    f.write(reporter.format_report_html(report))
```

API 的一个重要特性是函数调用树追踪。通过 `tracer.get_function_calls()` 方法可以获取完整的函数调用记录, 每个记录包含函数名、参数、返回值、调用深度和子调用列表。这使我们可以构建程序的动态调用图, 理解函数之间的调用关系和参数传递。以下代码展示了如何递归打印函数调用树

```
def print_call_tree(call, depth=0):
    indent = "  " * depth
    args = ", ".join(f"{k}={v}" for k, v in call.arguments.items())
    ret = f"->{call.return_value}" if call.return_value else ""
    print(f"{indent}{call.function_name}({args}){ret}")
    for child in call.children:
        print_call_tree(child, depth + 1)

# 打印顶层调用
for call in tracer.get_function_calls():
    print_call_tree(call)
```

4.3 支持的程序结构

PathTracer 能够分析 Python 程序中常见的控制流结构。顺序结构是最基本的程序结构, 代码按照从上到下依次执行, 不涉及分支或跳转。选择结构通过 `if/elif/else` 语句实现条件分支, 根据条件表达式的值选择不同的执行路径。循环结构包括 `for` 循环 (遍历可序列) 和 `while` 循环 (条件循环), 允许代码重复执行。异常处理结构使用 `try/except/else/finally` 来捕获和处理程序运行时可能出现的异常。嵌套结构是循环内包含条件语句, 或条件语句内包含循环的复杂组合。

这些结构在实际程序中经常混合使用, 例如一个 `for` 循环内部可能包含 `if` 条件判断来决定是否处理某个元素, 而这个 `if` 条件判断内部又可能包含 `while` 循环来执行某个操

作直到满足条件为止。

4.4 报告格式说明

文本格式报告以覆盖率信息以纯文本形式输出, 适合在终端中查看或或导入其他文档。格式如下

```
=====
EXECUTION PATH COVERAGE REPORT
=====
```

Total Branches: 22

Executed Branches: 16

Coverage: 72.7%

Branches by Type:

if : 17

for : 5

EXECUTED BRANCHES:

```
-----
target.py:
  Line 32: if [executed]
  Line 54: for [executed]
  ...
```

JSON 格式报告将数据序列化为 JSON 对象, 包含覆盖率百分比、总分支数、已执行分支数、按类型统计、各文件的详细执行信息。这种格式便于程序处理, 可以用于持续集成系统或自动化测试流程中以下是一个 JSON 报告的示例结构

```
{
  "coverage_percentage": 72.72727272727273,
  "total_branches": 22,
  "executed_branches": 16,
  "branches_by_type": {"if": 17, "for": 5},
  "file_reports": {
```

```

    "target.py": {
        "executed_lines": [1, 2, 3, 4, 5, ...],
        "executed_branches": [...]
    }
}
}

```

HTML 格式报告生成自包含的 HTML 文档, 使用 CSS 进行样式设计。报告页面采用响应式布局, 顶部显示覆盖率统计卡片, 中间显示按类型分类的统计信息, 下方展示源代码, 已执行的行显示为绿色背景, 未执行的行显示为红色背景。这种可视化方式使得用户可以直观地看到哪些代码被执行了哪些没有被执行。如果程序执行过程中有函数调用, 页面底部还会展示函数调用树。

4.5 扩展性设计

PathTracer 的模块化架构使其它易于扩展。添加新的分支类型支持只需要在 `models.py` 中定义新的枚举值并在 `cfg_builder.py` 中添加相应的识别逻辑。添加新的报告格式只需在 `reporter.py` 中添加新的格式化方法。添加新的追踪功能可以通过继承 `PathTracer` 类并重写回调方法实现。代码结构清晰, 各模块职责明确, 便于理解和维护。

4.6 本章小结

本章介绍了 PathTracer 的使用方法, 包括命令行接口和 Python API 两种方式。工具支持三种报告输出格式, 能够分析 Python 程序中常见的控制流结构。通过简洁的 API 设计, 用户可以轻松地将 PathTracer 集成到自己的开发和测试流程中。

结论

工作总结

本文设计并实现了一个 Python 程序执行路径追踪和代码覆盖率分析工具——Path-Tracer。该工具具有以下特点：

1. 技术实现

- 使用 Python 标准库 `sys.settrace` 实现运行时追踪，记录程序执行的每一行代码
- 使用 `ast` 模块进行静态分析，识别源代码中的所有控制流分支点
- 通过对比静态分支与动态执行路径，计算代码覆盖率

2. 功能特性

- 支持识别顺序、选择 (`if/elif/else`)、循环 (`for/while`)、异常处理 (`try/except`) 等程序结构
- 记录函数调用树，包含参数和返回值信息
- 支持文本、JSON、HTML 三种报告输出格式
- 提供命令行接口和 Python API 两种使用方式

3. 实验验证

通过完整的实验验证，工具成功：

- 设计并执行了 3 个测试用例 (TC1/TC2/TC3)，测试套件累计覆盖率达 72.7%
- 对 `demo.py` 进行了完整测试，覆盖了 `if/for/while/except` 等多种控制流结构
- 与 `coverage.py` 进行了对比分析 (使用 `-branch` 模式)，明确了两者统计口径差异
- 生成了 HTML 可视化报告 (见 `docs/coverage_demo_full.html`)

4. 工具定位

本实验验证了 PathTracer 在 Python 分支覆盖与执行路径跟踪上的可行性，适用于以下场景：

- **教学演示**：帮助理解程序控制流和执行路径
- **调试分析**：定位未覆盖的代码分支，辅助测试设计
- **代码审查**：记录函数调用关系和参数传递

局限性：本实验主要针对小型示例程序进行验证，尚未针对大型真实项目（如 Django、Flask 等框架）做系统评测。工具在大规模代码库上的性能和准确性有待进一步验证。

感谢张老师在课程设计过程中给予的悉心指导和帮助。

感谢同学们在开发过程中的讨论与建议，帮助完善了工具的功能设计。

感谢开源社区提供的 **Python** 标准库文档和示例，为本项目的开发提供了重要参考。

许同学

2026 年 4 月 2 日

附录 A 核心源代码

A.1 数据模型 (models.py)

```

1  """PathTracer 数据模型：定义执行路径追踪和覆盖率分析的核心数据结构"""
2
3  from dataclasses import dataclass, field
4  from typing import Set, Dict, List, Optional
5  from enum import Enum
6
7  # 枚举：分支类型，用于标识代码中的控制流结构
8  class BranchType(Enum):
9      SEQUENTIAL = "sequential" # 顺序执行
10     IF = "if"                # if 条件分支
11     ELSE = "else"            # else 分支
12     ELIF = "elif"            # elif 分支
13     FOR = "for"               # for 循环
14     WHILE = "while"           # while 循环
15     TRY = "try"               # try 异常处理块
16     EXCEPT = "except"       # except 异常捕获块
17
18 # 数据类：代码分支点
19 # 记录单个分支的位置、类型和执行状态
20 @dataclass
21 class CodeBranch:
22     file_path: str            # 源文件路径
23     line_no: int              # 分支起始行号
24     branch_type: BranchType   # 分支类型
25     end_line: int = 0         # 分支结束行号
26     is_executed: bool = False # 是否被执行过
27
28 # 数据类：函数调用记录
29 # 记录一次函数调用的完整信息，支持嵌套调用树
30 @dataclass
31 class FunctionCall:
32     function_name: str        # 函数名
33     file_path: str            # 所在文件

```



```

34     line_no: int                # 调用行号
35     call_depth: int            # 调用深度 (0=顶层)
36     arguments: Dict[str, str] = field(default_factory=dict) # 参数及值
37     return_value: Optional[str] = None # 返回值
38     children: List['FunctionCall'] = field(default_factory=list) # 子调用
39
40 # 数据类：执行路径
41 # 记录单个文件的完整执行轨迹
42 @dataclass
43 class ExecutionPath:
44     file_path: str                # 源文件路径
45     executed_lines: Set[int] = field(default_factory=set) # 已执行行号集合
46     executed_branches: Dict[int, CodeBranch] = field(default_factory=dict)
47                                     # 已执行分支
48     function_calls: List[FunctionCall] = field(default_factory=list) #
49                                     函数调用列表
50
51 # 数据类：覆盖率报告
52 # 汇总所有文件的覆盖率统计信息
53 @dataclass
54 class CoverageReport:
55     total_branches: int = 0                # 总分支数
56     executed_branches: int = 0            # 已执行分支数
57     branches_by_type: Dict[BranchType, int] = field(default_factory=dict)
58                                     # 按类型统计
59     coverage_percentage: float = 0.0      # 覆盖率百分比
60     file_reports: Dict[str, ExecutionPath] = field(default_factory=dict) #
61                                     各文件报告

```

Listing A.1 数据模型定义 (pathtracer/models.py)

A.2 运行时追踪器 (tracer.py)

```

1  """运行时追踪器：利用 Python 的 sys.settrace 机制追踪程序执行路径"""
2
3  import sys
4  from typing import Optional, Callable, Set, Dict, List

```

```

5 from .models import ExecutionPath, FunctionCall
6
7 class PathTracer:
8     """基于 sys.settrace 的 Python 程序执行路径追踪器"""
9
10    def __init__(self):
11        self.execution_paths: Dict[str, ExecutionPath] = {} #
            各文件的执行路径
12        self._original_trace: Optional[Callable] = None # 原始追踪函数
            (用于恢复)
13        self._target_file: Optional[str] = None # 目标追踪文件
14        self.function_calls: List[FunctionCall] = [] # 顶层函数调用列表
15        self._call_stack: List[FunctionCall] = [] # 函数调用栈
            (用于追踪嵌套)
16
17    def trace_file(self, file_path: str) -> 'PathTracer':
18        """设置要追踪的目标文件, 返回 self 支持链式调用"""
19        self._target_file = file_path
20        self.execution_paths[file_path] = ExecutionPath(file_path=file_path)
21        return self
22
23    def _trace_callback(self, frame, event: str, arg) ->
        Optional[Callable]:
24        """sys.settrace 的回调函数, 处理各类执行事件"""
25        if not self._target_file:
26            return None
27
28        # 只追踪目标文件中的事件
29        code = frame.f_code
30        if not code.co_filename.endswith(self._target_file):
31            return self._trace_callback
32
33        # 处理 'line' 事件: 记录执行的代码行
34        if event == 'line':
35            line_no = frame.f_lineno
36            self.execution_paths[self._target_file].executed_lines.add(line_no)
37

```

```

38     # 处理 'call' 事件：记录函数调用信息
39     elif event == 'call':
40         func_name = code.co_name
41         line_no = frame.f_lineno
42
43         # 从栈帧中提取函数参数
44         args = {}
45         try:
46             arg_count = code.co_argcount
47             arg_names = code.co_varnames[:arg_count]
48             for arg_name in arg_names:
49                 if arg_name in frame.f_locals:
50                     try:
51                         args[arg_name] = repr(frame.f_locals[arg_name])
52                     except Exception:
53                         args[arg_name] = '<unable to repr>'
54         except Exception:
55             pass
56
57         # 创建 FunctionCall 对象
58         call_depth = len(self._call_stack)
59         func_call = FunctionCall(
60             function_name=func_name,
61             file_path=code.co_filename,
62             line_no=line_no,
63             call_depth=call_depth,
64             arguments=args
65         )
66
67         # 将调用添加到父调用的 children 或顶层列表
68         if self._call_stack:
69             self._call_stack[-1].children.append(func_call)
70         else:
71             self.function_calls.append(func_call)
72             self.execution_paths[self._target_file].function_calls.append(func_call)
73
74         # 压入调用栈

```

```
75         self._call_stack.append(func_call)
76
77         # 处理 'return' 事件: 记录函数返回值
78         elif event == 'return':
79             if self._call_stack:
80                 func_call = self._call_stack.pop()
81                 if arg is not None:
82                     try:
83                         func_call.return_value = repr(arg)
84                     except Exception:
85                         func_call.return_value = '<unable to repr>'
86
87             return self._trace_callback
88
89     def __enter__(self):
90         """上下文管理器入口: 启动追踪"""
91         self._original_trace = sys.gettrace()
92         sys.settrace(self._trace_callback)
93         return self
94
95     def __exit__(self, exc_type, exc_val, exc_tb):
96         """上下文管理器出口: 停止追踪"""
97         sys.settrace(self._original_trace)
98         return False
99
100     def get_executed_lines(self, file_path: str) -> Set[int]:
101         """获取指定文件的已执行行号集合"""
102         if file_path in self.execution_paths:
103             return self.execution_paths[file_path].executed_lines
104         return set()
105
106     def get_function_calls(self) -> List[FunctionCall]:
107         """获取记录的所有函数调用"""
108         return self.function_calls
```

Listing A.2 运行时追踪器实现 (pathtracer/tracer.py)

A.3 控制流图构建器 (cfg_builder.py)

```

1  """控制流图构建器：通过静态分析 AST 提取代码中的所有分支点"""
2
3  import ast
4  from typing import List, Optional
5  from .models import CodeBranch, BranchType
6
7  class CFGBuilder:
8      """从 Python 源代码构建控制流图，识别所有分支结构"""
9
10     def __init__(self):
11         self.branches: List[CodeBranch] = [] # 提取的分支列表
12
13     def build_from_file(self, file_path: str) -> List[CodeBranch]:
14         """解析 Python 文件并提取所有分支点"""
15         with open(file_path, 'r', encoding='utf-8') as f:
16             source = f.read()
17
18         # 解析源代码为 AST
19         tree = ast.parse(source, filename=file_path)
20         self.branches = []
21
22         # 遍历 AST 查找控制流结构
23         for node in ast.walk(tree):
24             branch = self._extract_branch(file_path, node)
25             if branch:
26                 self.branches.append(branch)
27
28         return self.branches
29
30     def _extract_branch(self, file_path: str, node: ast.AST) ->
31         Optional[CodeBranch]:
32         """从 AST 节点提取分支信息"""
33
34         # if 语句
35         if isinstance(node, ast.If):
36             return CodeBranch(

```

```
36         file_path=file_path,
37         line_no=node.lineno,
38         branch_type=BranchType.IF,
39         end_line=self._get_end_line(node)
40     )
41
42     # for 循环
43     elif isinstance(node, ast.For):
44         return CodeBranch(
45             file_path=file_path,
46             line_no=node.lineno,
47             branch_type=BranchType.FOR,
48             end_line=self._get_end_line(node)
49         )
50
51     # while 循环
52     elif isinstance(node, ast.While):
53         return CodeBranch(
54             file_path=file_path,
55             line_no=node.lineno,
56             branch_type=BranchType.WHILE,
57             end_line=self._get_end_line(node)
58         )
59
60     # try/except 异常处理
61     elif isinstance(node, ast.ExceptHandler):
62         return CodeBranch(
63             file_path=file_path,
64             line_no=node.lineno,
65             branch_type=BranchType.EXCEPT,
66             end_line=self._get_end_line(node)
67         )
68
69     return None
70
71 def _get_end_line(self, node: ast.AST) -> int:
72     """获取代码块的最后一行行号"""
```

```

73     # Python 3.8+ 支持 end_lineno 属性
74     if hasattr(node, 'end_lineno') and node.end_lineno is not None:
75         return node.end_lineno
76     # 递归查找 body 中最大的行号
77     if hasattr(node, 'body') and node.body:
78         return max(self._get_end_line(n) for n in node.body)
79     return getattr(node, 'lineno', 0)
80
81     def get_branches_by_type(self, branch_type: BranchType) ->
82         List[CodeBranch]:
83         """按类型筛选分支"""
84     return [b for b in self.branches if b.branch_type == branch_type]

```

Listing A.3 控制流图构建器实现 (pathtracer/cfg_builder.py)

A.4 报告生成器 (reporter.py)

```

1  """覆盖率报告生成器：对比运行时追踪结果与静态 CFG 分析，生成覆盖率报告"""
2
3  import json
4  from typing import Dict, Any, List
5  from .models import CodeBranch, ExecutionPath, CoverageReport,
6      BranchType, FunctionCall
7  from .cfg_builder import CFGBuilder
8  from .tracer import PathTracer
9
10 class CoverageReporter:
11     """生成覆盖率报告：比较追踪路径与控制流图"""
12
13     def __init__(self):
14         self.cfg_builder = CFGBuilder() # 用于静态分析
15
16     def analyze(self, file_path: str, tracer: PathTracer) ->
17         CoverageReport:
18         """分析指定文件的覆盖率"""
19         # 静态分析：获取所有分支
20         all_branches = self.cfg_builder.build_from_file(file_path)

```

```
19
20     # 运行时数据：获取已执行的行
21     executed_lines = tracer.get_executed_lines(file_path)
22
23     # 标记已执行的分支
24     for branch in all_branches:
25         if branch.line_no in executed_lines:
26             branch.is_executed = True
27
28     # 计算统计数据
29     total = len(all_branches)
30     executed = sum(1 for b in all_branches if b.is_executed)
31
32     # 按分支类型统计
33     by_type: Dict[BranchType, int] = {}
34     for branch_type in BranchType:
35         type_branches = [b for b in all_branches if b.branch_type ==
36                           branch_type]
37         by_type[branch_type] = len(type_branches)
38
39     # 构建并返回覆盖率报告
40     return CoverageReport(
41         total_branches=total,
42         executed_branches=executed,
43         branches_by_type=by_type,
44         coverage_percentage=(executed / total * 100) if total > 0 else
45             0.0,
46         file_reports={file_path: ExecutionPath(
47             file_path=file_path,
48             executed_lines=executed_lines,
49             executed_branches={b.line_no: b for b in all_branches if
50                               b.is_executed}
51         )})
52
53 def format_report(self, report: CoverageReport) -> str:
54     """将覆盖率报告格式化为可读文本"""
```



```

53     lines = [
54         "=" * 60,
55         "EXECUTION PATH COVERAGE REPORT",
56         "=" * 60,
57         "",
58         f"Total Branches: {report.total_branches}",
59         f"Executed Branches: {report.executed_branches}",
60         f"Coverage: {report.coverage_percentage:.1f}%",
61         "",
62         "Branches by Type:",
63     ]
64
65     # 按类型输出分支统计
66     for branch_type, count in report.branches_by_type.items():
67         if count > 0:
68             lines.append(f" {branch_type.value:12s}: {count}")
69
70     lines.extend([
71         "",
72         "=" * 60,
73         "EXECUTED BRANCHES:",
74         "-" * 60,
75     ])
76
77     # 输出各文件的已执行分支详情
78     for file_path, exec_path in report.file_reports.items():
79         lines.append(f"\n{file_path}:")
80         for line_no, branch in
81             sorted(exec_path.executed_branches.items()):
82                 lines.append(
83                     f" Line {branch.line_no:4d}:
84                         {branch.branch_type.value:10s} [executed]"
85                 )
86
87     return "\n".join(lines)
88
89 def format_report_json(self, report: CoverageReport) -> str:

```

```

88     """将覆盖率报告格式化为 JSON (便于程序处理)"""
89     data: Dict[str, Any] = {
90         "coverage_percentage": report.coverage_percentage,
91         "total_branches": report.total_branches,
92         "executed_branches": report.executed_branches,
93         "branches_by_type": {
94             bt.value: count for bt, count in
95                 report.branches_by_type.items()
96         },
97         "file_reports": {},
98         "function_calls": []
99     }
100
101     # 构建各文件的报告数据
102     for file_path, exec_path in report.file_reports.items():
103         file_data: Dict[str, Any] = {
104             "executed_lines": sorted(list(exec_path.executed_lines)),
105             "executed_branches": []
106         }
107         for line_no, branch in
108             sorted(exec_path.executed_branches.items()):
109             file_data["executed_branches"].append({
110                 "line_no": branch.line_no,
111                 "branch_type": branch.branch_type.value,
112                 "end_line": branch.end_line
113             })
114         data["file_reports"][file_path] = file_data
115
116     # 包含函数调用信息
117     if exec_path.function_calls:
118         data["function_calls"] = self._serialize_function_calls(
119             exec_path.function_calls
120         )
121
122     return json.dumps(data, indent=2)
123
124 def _serialize_function_calls(self, calls: List[FunctionCall]) ->

```

```

List[Dict[str, Any]]:
123     """递归序列化函数调用树为 JSON 可用的字典列表"""
124     result = []
125     for call in calls:
126         call_dict = {
127             "function_name": call.function_name,
128             "file_path": call.file_path,
129             "line_no": call.line_no,
130             "call_depth": call.call_depth,
131             "arguments": call.arguments,
132             "return_value": call.return_value,
133             "children": self._serialize_function_calls(call.children) #
                                递归处理子调用
134         }
135         result.append(call_dict)
136     return result

```

Listing A.4 报告生成器实现 (pathtracer/reporter.py)

A.5 命令行接口 (cli.py)

```

1  """PathTracer 命令行接口：提供用户友好的命令行交互方式"""
2
3  import sys
4  import argparse
5  import io
6  from pathlib import Path
7  from .tracer import PathTracer
8  from .reporter import CoverageReporter
9
10 def main():
11     """CLI 主入口函数"""
12     raw_argv = sys.argv[1:]
13
14     # 无参数时显示帮助信息
15     if not raw_argv:
16         parser = argparse.ArgumentParser(

```

```

17         description='Trace Python program execution paths and generate
           coverage reports'
18     )
19     parser.add_argument('program', help='Python program file to trace')
20     parser.add_argument('--output', '-o', help='Output file for
           report', default=None)
21     parser.add_argument('--quiet', '-q', help='Only output report',
           action='store_true')
22     parser.add_argument(
23         '--format', '-f',
24         choices=['text', 'html', 'json'],
25         default='text',
26         help='Output format: text, html, or json (default: text)'
27     )
28     parser.add_argument('program_args', nargs='*', help='Arguments for
           target program')
29     parser.parse_args(raw_argv)
30     return
31
32     # 解析 CLI 参数与目标程序参数 (以第一个位置参数为分界)
33     cli_argv = []
34     program_args = []
35     program = None
36     index = 0
37     while index < len(raw_argv):
38         arg = raw_argv[index]
39         # 处理带值的选项
40         if arg in ('--output', '-o', '--format', '-f'):
41             if index + 1 >= len(raw_argv):
42                 cli_argv.append(arg)
43                 break
44             cli_argv.extend([arg, raw_argv[index + 1]])
45             index += 2
46             continue
47         # 处理标志选项
48         if arg in ('--quiet', '-q'):
49             cli_argv.append(arg)

```

```

50         index += 1
51         continue
52     # 第个位置参数是目标程序，后续为其参数
53     program = arg
54     cli_argv.append(arg)
55     program_args = raw_argv[index + 1:]
56     break
57
58 # 构建参数解析器
59 parser = argparse.ArgumentParser(
60     description='Trace Python program execution paths and generate
61         coverage reports'
62 )
63 parser.add_argument('program', help='Python program file to trace')
64 parser.add_argument('--output', '-o', help='Output file for report',
65     default=None)
66 parser.add_argument('--quiet', '-q', help='Only output report',
67     action='store_true')
68 parser.add_argument(
69     '--format', '-f',
70     choices=['text', 'html', 'json'],
71     default='text',
72     help='Output format: text, html, or json (default: text)'
73 )
74 parser.add_argument('program_args', nargs='*', help='Arguments for
75     target program')
76
77 args = parser.parse_args(cli_argv)
78
79 # 检查目标文件是否存在
80 if not Path(args.program).exists():
81     print(f"Error: File not found: {args.program}", file=sys.stderr)
82     sys.exit(1)
83
84 # 处理参数分隔符 --
85 if program_args[:1] == ['--']:
86     program_args = program_args[1:]

```

```
83
84 # 创建追踪器并设置目标文件
85 tracer = PathTracer().trace_file(args.program)
86
87 # 静默模式：捕获目标程序的 stdout
88 if args.quiet:
89     old_stdout = sys.stdout
90     sys.stdout = io.StringIO()
91
92 old_argv = sys.argv
93 reporter = CoverageReporter()
94
95 try:
96     # 设置 sys.argv 以便目标程序获取正确参数
97     sys.argv = [args.program] + program_args
98     # 在追踪上下文中执行目标程序
99     with tracer:
100         with open(args.program) as f:
101             code = compile(f.read(), args.program, 'exec')
102             exec(code, {'__name__': '__main__', '__file__':
103                        args.program})
104 finally:
105     # 恢复 sys.argv 和 stdout
106     sys.argv = old_argv
107     if args.quiet:
108         sys.stdout = old_stdout
109
110 # 生成覆盖率报告
111 report = reporter.analyze(args.program, tracer)
112
113 # 根据格式选择输出方法
114 if args.format == 'json':
115     output = reporter.format_report_json(report)
116 elif args.format == 'html':
117     output = reporter.format_report_html(report)
118 else:
119     output = reporter.format_report(report)
```

```

119
120     # 确定输出文件
121     output_file = args.output
122     if not output_file:
123         if args.format == 'html':
124             output_file = 'coverage-report.html'
125         elif args.format == 'json':
126             output_file = 'coverage-report.json'
127
128     # 输出报告
129     if output_file:
130         with open(output_file, 'w') as f:
131             f.write(output)
132         print(f"Report written to: {output_file}")
133     else:
134         print(output)
135
136 if __name__ == '__main__':
137     main()

```

Listing A.5 命令行接口实现 (pathtracer/cli.py)